PROTOCOL REFERENCE

# Universal Service Discovery Protocol (USDP)

USDP is a unified protocol for registering, discovering, querying, monitoring, and managing service records across mixed technical environments, ranging from APIs and databases to AI model endpoints, sensors, and network devices.

## What USDP Is

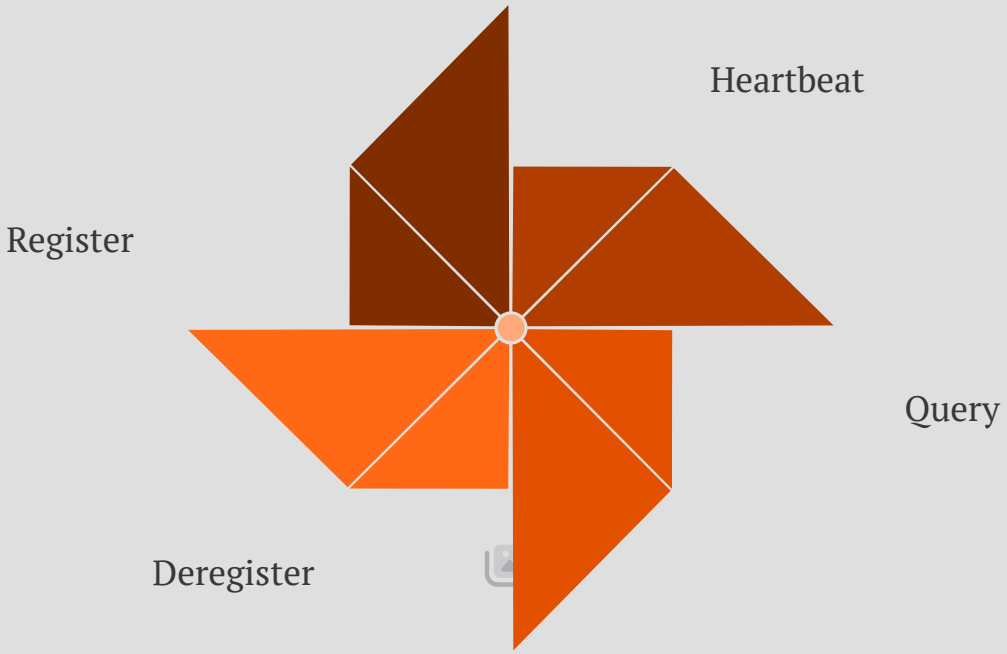
USDP is a service registry protocol that defines a contract between publishers and consumers. It standardizes how services announce their presence, how records are validated and persisted, and how operators locate and inspect services. It is not a service mesh, a load balancer, or a configuration management database; rather, it is an infrastructure-agnostic framework designed to cover the full operational surface of modern platforms.

### Core Coverage

- REST and gRPC APIs
- Databases and storage services
- Message brokers and event streams
- AI model endpoints
- Printers, cameras, sensors
- Network devices and appliances
- Internal platform services
- Third-party integration endpoints

## The Problem USDP Solves

Operational environments often outpace the tools used to track them. Teams rely on fragmented documentation, wikis, spreadsheets, and configuration files that quickly go stale and lack runtime health signals. USDP replaces this ad-hoc tracking with a protocol-governed registry where every entry is a structured record with a defined schema, publisher identity, and expiry mechanism. This ensures that only active services remain registered and that operators have an authoritative, current view of their infrastructure.




This lifecycle - Register, Heartbeat, Query, Deregister - is the operational foundation of USDP.

## Core Actors

### Registry
The central control plane that accepts registrations, validates records, persists state, enforces authentication, processes heartbeats, expires stale records, and serves discovery queries.

### Publishers
Systems, services, or devices that register service records. These may be native (self-registering) or non-native.

### SDK Clients
Application-layer integrations, including the Python library, used by developers to embed USDP publishing and discovery behavior into software.

### Agents
Non-native publisher proxies that observe and register systems that cannot self-publish, such as legacy services, sensors, or hardware.

### Operators
Platform engineers and administrators who use the dashboard for visibility, query execution, and registry health management.

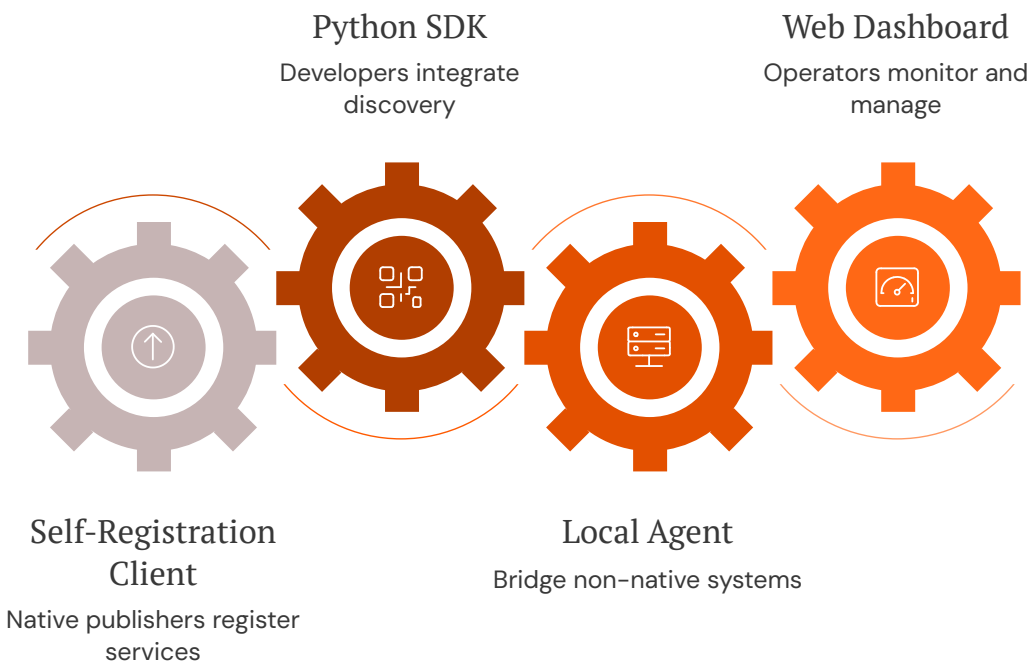
## Why This Matters Operationally

By standardizing service records, enforcing lifecycle discipline, and providing structured query access, USDP transforms service visibility from a maintenance burden into a reliable operational primitive. It replaces fragmented inventory with a single authoritative source of truth, offering heartbeat-driven freshness and criteria-driven discovery for modern environments.

ECOSYSTEM OVERVIEW

# The USDP Ecosystem: Five Tools, One Operating Model

USDP is implemented through five purpose-built tools, each designed for a specific role in the service discovery lifecycle. Together they form a complete ecosystem for service registration, discovery, and operational visibility.




| Tool | Role | Primary User | Primary Function |
|---|---|---|---|
| Registry | Control Plane | Platform Team | Accepts registrations, validates, persists, serves queries |
| Self-Registration Client | Native Publisher CLI | Services Publishing Themselves | Register, heartbeat, deregister via command line |
| Python SDK | Developer Library | Application Developers | Embed USDP behavior in Python code |
| Local Agent | Non-Native Publisher | Platform Team | Publish non-native systems via plugins |
| Web Dashboard | Operator UI | Platform Engineers, SREs | Browse, query, inspect, and administer registry operations |

COMPONENT DEEP DIVE

# Registry: The Control Plane

The registry is the authoritative backend of the USDP ecosystem. It is the single component that all other tools interact with.

### What It Does

- Accepts service record registrations
- Validates records against schema
- Persists state to SQLite
- Enforces role-aware authentication
- Processes heartbeats and expires stale records
- Serves discovery queries
- Exposes health, metrics, audit, retention, and purge admin capabilities
- Supports bounded v2 federation import and withdrawal

### How It Works

- HTTP server model
- Publisher ownership binding
- Request size limits
- Retention and purge handling
- Role-aware auth (publisher tokens, admin tokens, peer tokens)

## Running the Registry

The registry can be started using either of the following commands:

```
usdp-registry
python -m usdp_registry.main
```

## Important Environment Variables

| Variable | Purpose |
|---|---|
| `USDP_REGISTRY_HOST` | Bind host for the HTTP server |
| `USDP_REGISTRY_PORT` | Bind port for the HTTP server |
| `USDP_REGISTRY_DB` | Path to the SQLite database file |
| `USDP_PUBLISHER_TOKENS` | Tokens for publisher write operations |
| `USDP_ADMIN_TOKENS` | Tokens for admin endpoints |
| `USDP_PEER_TOKENS` | Tokens for federation paths |

## Who It Is For

Platform teams deploying the central service discovery infrastructure.

COMPONENT DEEP DIVE

# Self-Registration Client: The Native Publisher

The self-registration client is the primary tool for services that can publish themselves directly into the USDP registry. It is a standalone publisher CLI for native services.

## What It Does

- v2-by-default publisher HTTP client behavior
- Builders for shipped v2 service types
- Register-or-update helper behavior
- Retry support
- Client-side idempotency headers
- Background heartbeat worker with structured status snapshots

## When to Use It

Use the self-registration client when:

- Your service can run a CLI process
- You want to publish directly without embedding code
- You need simple register, heartbeat, deregister lifecycle
- You prefer command-line operation over library integration

## CLI Commands

```
usdp-client register <service_file>
usdp-client register-or-update <service_file>
usdp-client update <service_file>
usdp-client heartbeat <service_id>
usdp-client run-heartbeat <service_id>
usdp-client deregister <service_id>
```

## Configuration

Important environment variables:

- `USDP_REGISTRY_URL`
- `USDP_PUBLISHER_TOKEN`
- `USDP_PROTOCOL_VERSION`
- `USDP_CLIENT_TIMEOUT`
- `USDP_CLIENT_RETRIES`
- `USDP_CLIENT_RETRY_DELAY`

## Lifecycle Pattern

1. Configure environment variables
2. Create service definition file
3. Register: `usdp-client register <service_file>`
4. Heartbeat: `usdp-client run-heartbeat <service_id>`
5. Deregister: `usdp-client deregister <service_id>`

## Who It Is For

Services that can run a CLI process and want direct registry integration without code changes.

COMPONENT DEEP DIVE

# Python SDK: The Application Developer Integration

The Python SDK is the library-first integration path for developers embedding USDP behavior directly into Python applications. Where the self-registration client is a CLI tool, the SDK is a library.

## What It Provides

- Publish lifecycle operations (register, update, heartbeat, deregister)
- v1 and v2 discovery operations
- Typed models for services, provenance, publisher identity, criteria, and paged query results
- Iterator and page helpers
- Builder functions for shipped v2 service types
- Optional admin helpers for metrics, audit, retention, and purge

## Core Classes and Functions

### Main Classes

- USDPSDK: main client class
- SDKConfig: configuration object
- QueryCriterion: query builder

### Builder Functions

- build_api_service
- build_database_service
- build_storage_service
- build_message_broker_service
- build_ai_model_endpoint_service
- build_printer_service
- build_camera_service
- build_sensor_service

### Lifecycle & Discovery

**Lifecycle methods:** register_service, register_or_update_service, update_service, heartbeat, deregister_service

**Discovery methods:** query_services, query_service_records, iter_services, get_service

### Operational & Admin Methods

**Operational methods:** get_health

**Admin methods:** get_metrics, get_audit_events, get_retention, purge_due_records

## Example Usage

```
from usdp_sdk import SDKConfig, USDPSDK, QueryCriterion

sdk = USDPSDK(SDKConfig(
    registry_url="http://127.0.0.1:8000",
    protocol_version="2.0",
))

result = sdk.query_service_records(
    criteria=QueryCriterion.predicate("service_type", "eq", "database"),
)
```

## When to Use It

Use the Python SDK when:

- You are building a Python application
- You want to embed service discovery or publishing directly in code
- You need programmatic access to USDP operations
- You want typed models and builder utilities

## Who It Is For

Python application developers building USDP-aware services.

COMPONENT DEEP DIVE

# Local Agent: The Non-Native Publisher Path

The local agent exists to solve a specific and common infrastructure problem: many of the most operationally significant systems on a network - printers, cameras, sensors, legacy services - cannot self-publish. The local agent brings them into USDP.

## What It Does

- Plugin-driven discovery and publishing
- State tracking for register, update, heartbeat, and deregister decisions
- Optional per-cycle reporting
- Supports both run-once and looped execution
- Observes non-native systems and manages their service records on their behalf

## Plugin Model

The agent uses a plugin architecture. Built-in plugin types:

- `api`: for configured HTTP API endpoints with health checks
- `printer`: for network printers
- `camera`: for network cameras
- `sensor`: for environmental and IoT sensors

Each plugin handles discovery and state management for its service type.

## State Tracking

The agent maintains state to make intelligent decisions about:

- When to register a newly discovered service
- When to update an existing service record
- When to send heartbeats
- When to deregister a service that has gone offline

## CLI Commands

```
usdp-agent run-once <config_file>
usdp-agent run-loop <config_file>
```

- **run-once**: execute one discovery and publish cycle, then exit
- **run-loop**: execute discovery and publish cycles continuously at configured interval

## Configuration

The agent uses JSON configuration. Key concepts:

- `registry_url`: where to publish
- `publisher_token`: authentication token
- `publisher_name`: identity for published records
- `protocol_version`: USDP version (typically "2.0")
- `state_file`: where to persist state between runs
- `report_file`: optional per-cycle reporting
- `cycle_interval_seconds`: interval for run-loop mode
- The `plugins` field is an object (dictionary) keyed by plugin type, with each key containing that plugin's configuration. Built-in plugin types include `api`, `printer`, `camera`, and `sensor`.

## Who It Is For

Platform teams managing non-native systems (printers, cameras, sensors, legacy services) that need to be discoverable in USDP.

COMPONENT DEEP DIVE

# Web Dashboard: The Operator Surface

The web dashboard is the visual operator interface for the USDP ecosystem. It provides a browser-based view into the registry and enables operators to browse, query, inspect, and administer registry visibility and operations.

## What It Does

- v2-by-default registry browsing
- Criteria-driven discovery queries
- Type-aware detail rendering
- Same-origin proxying to registry read and admin endpoints
- Admin metrics, audit, retention, and purge views when admin token is configured

## Core Capabilities

### Browse

View all registered services in the registry

### Query

Use criteria-driven discovery to find services by type, owner, status, or other attributes

### Inspect

View detailed service records with full metadata

### Administer

Access metrics, audit logs, retention policies, and purge operations when admin access is configured

## Running the Dashboard

Command:

```
usdp-dashboard
```

Important environment variables:

- `USDP_DASHBOARD_HOST`
- `USDP_DASHBOARD_PORT`
- `USDP_REGISTRY_URL`
- `USDP_PROTOCOL_VERSION`
- `USDP_ADMIN_TOKEN` (optional, enables admin views)

## How It Connects to the Registry

The dashboard communicates with the registry via HTTP. It proxies read and admin requests to the registry's endpoints. When an admin token is configured, the dashboard can access metrics, audit logs, retention settings, and purge operations.

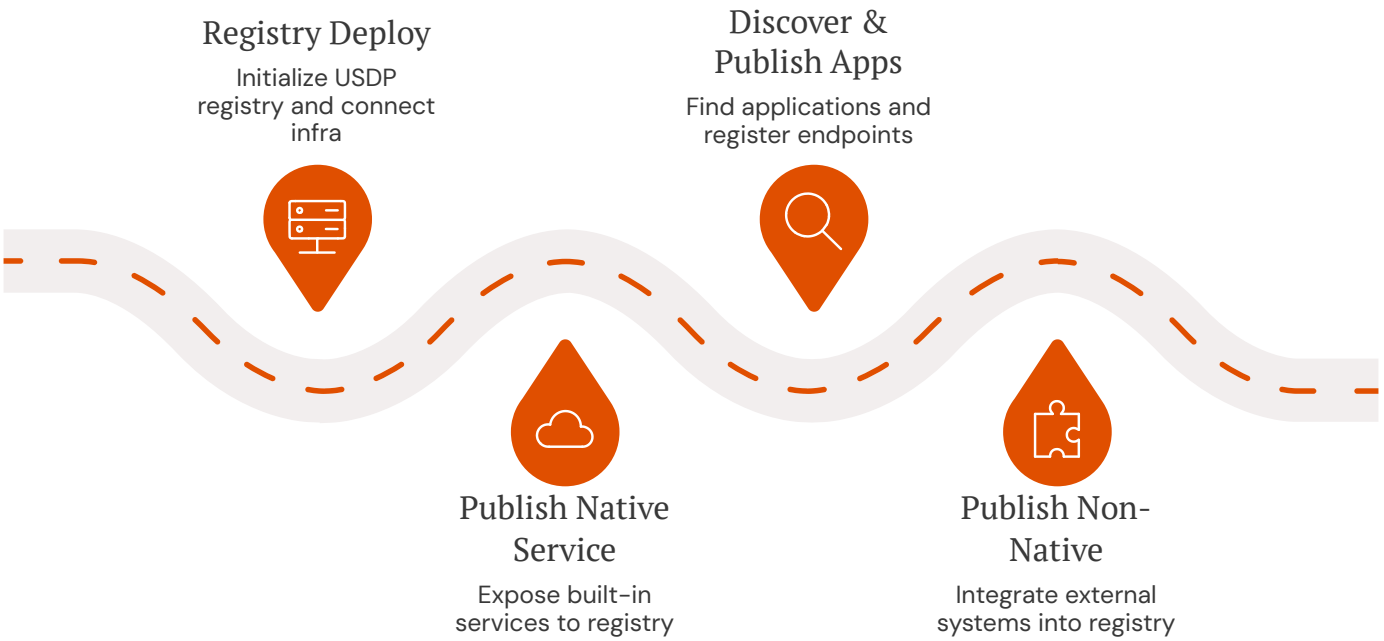
## Who It Is For

Platform engineers, SREs, and administrators who need operational visibility and access to registry administration features.

END-TO-END WORKFLOW

# End-to-End Operational Workflow

The following walkthrough illustrates how the five USDP tools work together in a realistic operational sequence. This represents a complete deployment from infrastructure setup through operational visibility.




01

## Registry Deployment

- Platform team deploys the registry
- Configures auth tokens (publisher, admin, peer)
- Registry becomes the authoritative service record store

02

## Native Service Publishing

- A service that can self-publish starts up
- Uses self-registration client to register itself
- Runs heartbeat loop to keep record fresh
- Registry accepts and validates the record

03

## Application Discovery and Publishing

- A Python application starts
- Uses Python SDK to query for database services
- Finds the registered database service
- Application publishes its own service record via SDK
- Other applications can now discover it

04

## Non-Native System Publishing

- Platform team deploys local agent with printer and camera plugins
- Agent discovers printers and cameras on the network
- Agent publishes them to registry on their behalf
- Operators can now see and query these devices

05

## Operator Visibility

- Platform engineer opens web dashboard
- Browses all registered services (native and non-native)
- Runs criteria-driven queries to find services by type
- Inspects service details and metadata

06

## Administrative Operations

- Admin reviews audit logs via dashboard
- Checks retention policies
- Purges expired records
- Monitors registry health and metrics

All five tools work together to create a complete service discovery ecosystem. Services publish themselves or are published by agents. Operators gain unified visibility. The registry remains the authoritative source of truth.

DECISION GUIDE

# Tool Selection Guide: Which Tool Should I Use?

Selecting the right tool for a given context is straightforward once the role of each tool is clear. Use this guide to determine which tool fits your situation.

## Decision Matrix by Role

| Situation | Best Tool | Why |
|---|---|---|
| Your service can run a CLI process and needs to publish itself directly | **Self-Registration Client** | Simple command-line operation, no code changes required, built-in heartbeat support |
| You are building a Python application and want to embed service discovery or publishing | **Python SDK** | Programmatic access, typed models, builder utilities, full lifecycle control |
| You have non-native systems (printers, cameras, sensors) that cannot self-publish | **Local Agent** | Plugin-driven discovery, state tracking, handles systems that cannot run code |
| You need to see all registered services and run discovery queries | **Web Dashboard** | Visual interface, criteria-driven queries, type-aware rendering, admin operations |
| You are deploying the central service discovery infrastructure | **Registry** | Control plane, authoritative backend, all other tools depend on it |

## SDK vs Self-Registration Client

Both the Python SDK and self-registration client can publish services. Choose based on:

- **SDK:** when you are writing application code and want programmatic control
- **Self-Registration Client:** when you have a standalone service that can run a CLI process

## When to Use Local Agent

Use the local agent when:

- You have systems that cannot run code or CLI processes
- You need to discover and publish devices (printers, cameras, sensors)
- You want plugin-driven extensibility
- You need state tracking across discovery cycles

Do not use the local agent when:

- Your service can self-publish via SDK or CLI
- You only need to query the registry (use dashboard instead)

## The Registry Is Always Required

The registry is the foundation. All other tools depend on it. Deploy the registry first, then add other tools as needed.

ADOPTION SUMMARY

# Practical Adoption: From First Deployment to Full Stack

Adopting USDP does not require a big-bang deployment. The ecosystem is designed to be adopted incrementally, starting with a single registry and expanding as operational needs grow.

### Phase 1: Foundation

Deploy the registry:

- Start `usdp-registry`
- Configure auth tokens
- Verify HTTP endpoints are accessible
- This is your single source of truth

### Phase 2: Native Publishers

Add services that can self-publish:

- Deploy self-registration client for services that can run CLI processes
- Or embed Python SDK in applications
- Services begin registering themselves
- Heartbeat loops keep records fresh

### Phase 3: Operator Visibility

Deploy web dashboard:

- Start `usdp-dashboard`
- Point it to your registry
- Operators can now browse and query services
- No code changes needed

### Phase 4: Non-Native Systems

Deploy local agent:

- Configure plugins for printers, cameras, sensors, legacy systems
- Agent discovers and publishes them
- Operators see complete infrastructure picture

### Phase 5: Administrative Operations

Enable admin features:

- Configure admin token on dashboard
- Review audit logs
- Manage retention policies
- Purge expired records
- Monitor metrics

## Incremental Growth

You do not need all five tools immediately. Start with registry and one publisher type. Add tools as operational needs grow. Each tool is independent and can be deployed on its own schedule.

## Conclusion

USDP is designed for incremental adoption. Begin with the registry, add publishers as services come online, deploy the dashboard for visibility, and add the agent for non-native systems. The result is a unified, authoritative service discovery system that grows with your infrastructure.